



Modularización de proyectos

El objetivo aquí no es “hacer muchos archivos” porque sí. Es aprender dos ideas que van juntas: **cada pieza del sistema vive en un lugar claro y la sesión guarda solo lo que ya pasó validación**. Nos vamos a centra en hacer separación de responsabilidades. Esto quiere decir que cada pieza del sistema vive en un lugar claro y sirve para una sola cosa. Luego el sistema funciona como un todo.

Eso hace que el código sea:

- más fácil de leer
- más fácil de cambiar
- más fácil de probar
- preparado para usar **historial como contexto** (Unidad 2+)

Objetivos

Modularización

- Separar `config`, `prompts`, `logic` y `state`.
- Reducir el `main` a orquestación mínima.
- Evitar “prompts hardcodeados” y constantes repetidas.
- Entender **quién importa a quién** (sin dependencias circulares).

Estado en memoria

- Definir un `state` con `dict` y `list`.
- Guardar registros validados en un historial (solo si `status == "ok"`).
- Listar, filtrar y contar registros en sesión.

El **registro de ejemplo** que usamos en toda la unidad:

```
registro = {  
    "nombre": "Ana",  
    "email": "ana@ejemplo.com",  
    "codigo": "TKT-1234",  
    "tipo": "summary",      # summary | translate | qa
```

```
"prioridad": 3,          # entero 1-5
}
```

1) De script monolítico a módulos

1.1 El problema (todo en un sitio)

Cuando todo está en una celda o en un solo `.py`, suele pasar esto:

- cambias `VALID_TIPOS` en un sitio y se te olvida otro
- un mensaje de error está copiado tres veces
- no sabes si una función "valida" o "imprime" o "guarda historial"

Ejemplo **monolítico** (anti-patrón):

```
# --- constantes mezcladas con lógica y prints ---
VALID_TIPOS = ["summary", "translate", "qa"]
MSG_ERROR = "Input inválido"

def validar_registro(datos):
    errores = []
    if not str(datos.get("nombre", "")).strip():
        errores.append("Nombre inválido: no puede estar vacío.")
    tipo = str(datos.get("tipo", "")).strip().lower()
    if tipo not in VALID_TIPOS:
        errores.append(f"Tipo inválido. Usa una de: {VALID_TIPOS}")
    return errores

def procesar_registro(datos):
    errores = validar_registro(datos)
    if errores:
        return {"status": "error", "mensaje": MSG_ERROR, "data": {"errores":
errores}}
    return {"status": "ok", "mensaje": "Registro válido", "data": datos}

datos = {"nombre": "", "tipo": "resumen", "prioridad": 3}
r = procesar_registro(datos)
print "[" + r["status"].upper() + "]", r["mensaje"] # presentación mezclada
```

Funciona una vez. **No escala** cuando añades email, regex, historial o prompts.

1.2 La solución (misma lógica, sitios distintos)

Mismo comportamiento, repartido en archivos con nombres estándar. Este es un ejemplo de cómo podría ser la estructura de un proyecto. Con esto hacemos la separación de responsabilidades.

```
mini_proyecto/
├── config.py      # reglas fijas
```

```
|— prompts.py      # textos reutilizables
|— logic.py        # validar + procesar + respuestas
|— state.py        # sesión: historial, contadores, operaciones
|— main.py         # orquestación + prints
```

2) Qué va en cada módulo

config.py

- constantes y límites (longitudes mínimas, rangos)
- listas de opciones válidas (`VALID_TIPOS`)
- patrones regex (`PATRON_EMAIL`, `PATRON_CODIGO`, ...) si los usas del bloque 1
- placeholders futuros (por ejemplo, `MODEL = "gemini-..."`) **sin llamar a la API**

Regla: si es un valor que “casi nunca cambia en runtime”, va aquí.

prompts.py

- plantillas de texto
- mensajes de salida reutilizables
- prompts (cuando lleguemos a Unidad 2) **sin pegarlos en medio del código**

Regla: si es texto que un humano edita a menudo, va aquí — no en `logic.py`.

logic.py

- validaciones (`validar_registro`)
- decisiones de flujo (`procesar_registro`, handlers)
- helpers de respuesta (`respuesta_ok`, `respuesta_error`)

Regla: funciones puras de negocio: reciben datos, devuelven dicts. **Sin print.**

state.py

- datos que **cambian** en la sesión:
 - historial en memoria (`list` de registros)
 - contadores (`dict` de frecuencias)
 - última acción
- funciones para **guardar solo si OK**, listar, filtrar y contar (Parte B)

Regla: `logic` valida y decide; `state` **recuerda** lo que ya pasó el filtro. No mezcles validación larga dentro de `state.py`.

main.py (o última celda del notebook)

- importa módulos y ejecuta el flujo
- imprime o muestra resultados (`imprimir_respuesta`)
- **no** contiene reglas de validación ni constantes largas

3) Reglas de imports (quién puede importar a quién)



Syntax error in text

mermaid version 11.16.0

Módulo	Puede importar	No debería importar
<code>config.py</code>	(nada, o solo <code>re</code> para compilar regex)	<code>logic</code> , <code>main</code>
<code>prompts.py</code>	(nada)	<code>logic</code> , <code>main</code>
<code>logic.py</code>	<code>config</code> , <code>prompts</code>	<code>main</code>
<code>state.py</code>	<code>config</code> (opcional, límites)	<code>main</code> , <code>logic</code> (evitar acoplar)
<code>main.py</code>	<code>logic</code> , <code>state</code> , <code>prompts</code>	no redefinir validaciones aquí

Dependencia circular = `logic` importa `main` y `main` importa `logic` → se rompe al importar. Por eso `main` solo **orquesta**.

4) Ejemplo completo por archivos

📁 **Proyecto ejecutable (copia lista):** `03_Modularización_de_proyectos_ejemplo/` — desde esa carpeta:
`python main.py`

También puedes copiar los fragmentos a tus propios `.py` o reproducirlos en celdas del notebook de ejemplos.

4.1 `config.py`

```
# config.py – reglas del juego (sin lógica de negocio)

VALID_TIPOS = ["summary", "translate", "qa"]

PRIORIDAD_MIN = 1
PRIORIDAD_MAX = 5

# Opcional: del bloque 1 (regex). Aquí solo definimos el patrón.
import re

PATRON_CODIGO = re.compile(r"^TKT-\d{4}$")

# Placeholder para Unidad 2 (sin llamar a la API en esta unidad)
MODEL = "gemini-proximamente"
```

4.2 prompts.py

```
# prompts.py – textos editables sin tocar la lógica

MSG_ERROR_INPUT = "Input inválido"
MSG_REGISTRO_OK = "Registro válido"

PLANTILLA_TAREA = "Tarea '{tipo}' con prioridad {prioridad}."

# Ejemplo de plantilla para cuando haya LLM (Unidad 2)
PLANTILLA_PROMPT_RESUMEN = (
    "Resume el siguiente texto de forma breve:\n\n{texto}"
)
```

4.3 logic.py

```
# logic.py – validar, decidir, devolver dict estructurado

from config import PATRON_CODIGO, PRIORIDAD_MAX, PRIORIDAD_MIN, VALID_TIPOS
from prompts import MSG_ERROR_INPUT, MSG_REGISTRO_OK, PLANTILLA_TAREA

def respuesta_ok(mensaje: str, data: dict | None = None) -> dict:
    return {"status": "ok", "mensaje": mensaje, "data": data or {}}

def respuesta_error(mensaje: str, errores: list[str]) -> dict:
    return {"status": "error", "mensaje": mensaje, "data": {"errores": errores}}

def validar_registro(datos: dict) -> list[str]:
    errores: list[str] = []

    if not isinstance(datos, dict):
        return ["El registro debe ser un dict."]

    if not str(datos.get("nombre", "")).strip():
        errores.append("Nombre inválido: no puede estar vacío.")

    codigo = str(datos.get("codigo", "")).strip()
    if codigo and PATRON_CODIGO.fullmatch(codigo) is None:
        errores.append("Código inválido: usa formato TKT-1234.")

    tipo = str(datos.get("tipo", "")).strip().lower()
    if tipo not in VALID_TIPOS:
        errores.append(f"Tipo inválido. Usa una de: {VALID_TIPOS}")

    prioridad = datos.get("prioridad")
    if not isinstance(prioridad, int) or not (PRIORIDAD_MIN <= prioridad <=
PRIORIDAD_MAX):
```

```

        errores.append(
            f"Prioridad inválida: debe ser un entero {PRIORIDAD_MIN}-{PRIORIDAD_MAX}."
        )

    return errores

def procesar_registro(datos: dict) -> dict:
    errores = validar_registro(datos)
    if errores:
        return respuesta_error(MSG_ERROR_INPUT, errores)

    tipo = str(datos.get("tipo", "")).strip().lower()
    prioridad = datos.get("prioridad")
    mensaje = PLANTILLA_TAREA.format(tipo=tipo, prioridad=prioridad)

    return respuesta_ok(
        MSG_REGISTRO_OK,
        data={"mensaje_tarea": mensaje, "tipo": tipo, "prioridad": prioridad},
    )

```

4.4 state.py

```

# state.py – sesión en memoria (historial + operaciones)

def inicializar_estado() -> dict:
    return {
        "historial": [],          # lista de registros dict (solo los OK)
        "contadores": {},        # p. ej. {"summary": 2, "qa": 1}
        "ultima_accion": None,
    }

def guardar_si_ok(state: dict, datos: dict, resultado: dict) -> bool:
    """Solo añade al historial si la lógica devolvió status ok."""
    if resultado.get("status") != "ok":
        state["ultima_accion"] = "rechazado"
        return False

    entrada = {**datos, "procesado": resultado.get("data", {})}
    state["historial"].append(entrada)

    tipo = str(datos.get("tipo", "")).strip().lower()
    state["contadores"][tipo] = state["contadores"].get(tipo, 0) + 1
    state["ultima_accion"] = "guardado"
    return True

def listar_historial(state: dict) -> list[dict]:

```

```

    return list(state.get("historial", []))

def filtrar_por_tipo(state: dict, tipo: str) -> list[dict]:
    t = (tipo or "").strip().lower()
    return [
        r
        for r in state.get("historial", [])
        if str(r.get("tipo", "")).strip().lower() == t
    ]

def filtrar_prioridad_minima(state: dict, minimo: int) -> list[dict]:
    return [
        r
        for r in state.get("historial", [])
        if isinstance(r.get("prioridad"), int) and r["prioridad"] >= minimo
    ]

def contar_por_tipo(state: dict) -> dict:
    return dict(state.get("contadores", {}))

def vaciar_historial(state: dict) -> None:
    state["historial"].clear()
    state["contadores"].clear()
    state["ultima_accion"] = "vaciado"

def ultimos_n(state: dict, n: int = 3) -> list[dict]:
    """Útil más adelante para armar contexto con los últimos N registros."""
    historial = state.get("historial", [])
    return historial[-n:] if n > 0 else []

```

4.5 main.py

```

# main.py – orquestación mínima (sí puede usar print)

from logic import procesar_registro
from state import (
    contar_por_tipo,
    filtrar_por_tipo,
    guardar_si_ok,
    inicializar_estado,
    listar_historial,
    ultimos_n,
)

def imprimir_respuesta(r: dict) -> None:

```

```

status = r.get("status", "unknown").upper()
mensaje = r.get("mensaje", "")
print(f"[{status}] {mensaje}")

if r.get("status") == "error":
    for e in r.get("data", {}).get("errores", []):
        print(" -", e)
elif r.get("status") == "ok" and "mensaje_tarea" in r.get("data", {}):
    print(" →", r["data"]["mensaje_tarea"])

def main() -> None:
    state = inicializar_estado()

    casos = [
        {"nombre": "Ana", "codigo": "TKT-1234", "tipo": "summary", "prioridad":
3},
        {"nombre": "Luis", "codigo": "TKT-5678", "tipo": "qa", "prioridad": 5},
        {"nombre": "", "tipo": "resumen", "prioridad": "mucho"},
    ]

    for datos in casos:
        resultado = procesar_registro(datos)
        imprimir_respuesta(resultado)
        guardar_si_ok(state, datos, resultado)

    print("\n--- Historial (solo OK) ---")
    for i, reg in enumerate(listar_historial(state), start=1):
        print(f" {i}. {reg.get('nombre')} · {reg.get('tipo')} · prioridad
{reg.get('prioridad')}")

    print("\n--- Solo tipo 'summary' ---")
    for reg in filtrar_por_tipo(state, "summary"):
        print(" ", reg.get("nombre"), reg.get("codigo"))

    print("\n--- Contadores por tipo ---", contar_por_tipo(state))
    print("--- Últimos 2 (borrador de contexto) ---", ultimos_n(state, 2))

if __name__ == "__main__":
    main()

```

Salida esperada (aproximada):

```

[OK] Registro válido
→ Tarea 'summary' con prioridad 3.
[OK] Registro válido
→ Tarea 'qa' con prioridad 5.
[ERROR] Input inválido
- Nombre inválido: no puede estar vacío.
...

```



```

--- Historial (solo OK) ---
1. Ana · summary · prioridad 3
2. Luis · qa · prioridad 5

--- Solo tipo 'summary' ---
Ana TKT-1234

--- Contadores por tipo --- {'summary': 1, 'qa': 1}

```

5) Persistencia en memoria

En esta unidad “**persistencia**” significa:

guardar datos mientras el programa está en ejecución (en memoria).

En Colab/Jupyter, esa memoria **se pierde al reiniciar el kernel**. No es una base de datos: basta para aprender sesión, historial y operaciones con `list` y `dict` — la misma idea que luego usarás como **contexto** para un LLM.

5.1 Patrón: validar → procesar → guardar

Orden recomendado en `main` (nunca al revés):

1. `resultado = procesar_registro(datos)` — valida y decide en `logic.py`.
2. `guardar_si_ok(state, datos, resultado)` — solo hace `append` si `status == "ok"`.

Así el historial **no contamina** entradas inválidas. Los errores se muestran al usuario, pero no se “recuerdan” como si fueran datos buenos.

5.2 Qué guardamos en cada entrada

Cada elemento del historial es un `dict` (el registro original + metadatos opcionales):

```

entrada = {**datos, "procesado": resultado.get("data", {})}

```

Eso permite, más adelante, reconstruir qué se hizo con cada registro válido sin volver a llamar a `logic`.

5.3 Operaciones frecuentes

Operación	Función	Idea
Listar todo	<code>listar_historial(state)</code>	Recorrer e imprimir o exportar
Filtrar por tipo	<code>filtrar_por_tipo(state, "summary")</code>	List comprehension sobre <code>historial</code>
Filtrar por prioridad	<code>filtrar_prioridad_minima(state, 4)</code>	Solo urgentes

Operación	Función	Idea
Contar	<code>contar_por_tipo(state)</code>	Dict de frecuencias actualizado al guardar
Vaciar sesión	<code>vaciar_historial(state)</code>	<code>historial.clear()</code> + reset contadores
Últimos N	<code>ultimos_n(state, 3)</code>	Base para "contexto reciente"

5.4 Puente a Context Engineering

Ese historial en memoria es el paso natural para:

- construir **contexto** (texto que acompaña al prompt)
- incluir solo los **últimos N** registros (`ultimos_n`)
- generar un resumen de sesión a partir de datos ya validados

En el futuro harás lo mismo con prompts reales y API; aquí vemos el esqueleto es **Python + estado**, sin LLM.

5.5 Dónde tocar si cambias el comportamiento del historial

Qué cambias	Toca
"No guardar si prioridad < 3"	<code>guardar_si_ok</code> en <code>state.py</code> (o regla en <code>logic</code> antes de devolver <code>ok</code>)
Formato de lo guardado	<code>guardar_si_ok</code>
Cómo se listan en consola	<code>main.py</code>
Reglas de validación	<code>logic.py</code> (no <code>state</code>)

6) Señal de buen diseño: cambiar en un solo sitio

La idea de separacion de responsabilidades te ayudará a que en el futuro cambies una sola cosa sin tener que tocar todo el proyecto.

Qué cambias	Toca solo
Rango de prioridad (1-5 → 1-10)	<code>config.py</code> (<code>PRIORIDAD_MAX</code>)
Tipos válidos (<code>qa</code> → <code>chat</code>)	<code>config.py</code> (<code>VALID_TIPOS</code>)
Texto "Input inválido"	<code>prompts.py</code> (<code>MSG_ERROR_INPUT</code>)
Regla "nombre obligatorio"	<code>logic.py</code> (<code>validar_registro</code>)
Cómo se imprime en consola	<code>main.py</code> (<code>imprimir_respuesta</code>)

Ejercicio mental: sube `PRIORIDAD_MAX` a 10 en `config.py`. Un registro con `prioridad: 8` debe pasar **sin** editar `validar_registro` (solo usa `PRIORIDAD_MIN` / `PRIORIDAD_MAX` importados).

7) `if __name__ == "__main__"`

Cuando tengas scripts `.py`, usa este patrón para que:

- se pueda **importar** el módulo sin ejecutar el programa
- el programa solo corra si ejecutas el archivo directamente

```
def main() -> None:  
    ...  
  
if __name__ == "__main__":  
    main()
```

Así puedes hacer `from logic import procesar_registro` en un test o en un notebook **sin** que se ejecute todo el flujo al importar.